MAZI Fraud AI Dashboard

Complete Developer Guide & Project Roadmap

Final Exam Project Master Plan

June 23, 2026

Contents

1 Project Architecture & Tech Stack Overview **1**

2 VS Code Environment Setup **2**
2.1 Required VS Code Extensions 2
2.2 Terminal Installations (pip install) 2

3 Step-by-Step Implementation Guide **2**
3.1 Step 1: Fix & Train the Machine Learning Model 2
3.2 Step 2: MySQL Database Setup 3
3.3 Step 3: Integrating C++ for High Performance 3
3.4 Step 4: Upgrading your Flask Backend (`app.py`) 3
3.5 Step 5: Building the Frontend Dashboard 3

4 Recommendations & What to Add/Remove **4**
4.1 What to Add 4
4.2 What to Remove / Reconsider 4

5 Conclusion for Deployment **4**

1 Project Architecture & Tech Stack Overview

The MAZI Fraud AI Dashboard is a brilliant concept for a final year project. It combines web development, machine learning, and database management. Here is how your requested technologies fit together perfectly:

- **Frontend (User Interface):** HTML, CSS, and **JavaScript** (Note: JavaScript, not Java, is used alongside HTML/CSS to make web pages interactive, like rendering charts and updating the dashboard in real-time).
- **Backend (Server & API):** Python using the Flask framework. This serves your web pages and acts as the bridge between your frontend, your database, and your AI model.
- **Machine Learning:** Python using Scikit-Learn and Pandas to train a Logistic Regression model on your `AIML Dataset.csv`.

- **High-Performance Logic (C++):** C++ can be used to write highly optimized algorithms (e.g., rapid anomaly scoring or massive dataset filtering). This C++ code can be called directly from Python using `pybind11`.
- **Database:** MySQL to store transaction histories, confirmed fraud cases, and user logs.

2 VS Code Environment Setup

To build this seamlessly, you need the right tools installed in your VS Code environment.

2.1 Required VS Code Extensions

Search for these in the VS Code Extensions tab and install them:

- **Python** (by Microsoft) - For Python IntelliSense and debugging.
- **C/C++** (by Microsoft) - For C++ syntax highlighting and compilation.
- **Jupyter** (by Microsoft) - To run your `.ipynb` ML files inside VS Code.
- **SQLTools** & **SQLTools MySQL/MariaDB** - To view and manage your MySQL database directly inside VS Code.

2.2 Terminal Installations (pip install)

Open your VS Code terminal and run the following commands to install your Python dependencies:

```
# 1. Core Data Science & ML
pip install pandas numpy scikit-learn matplotlib seaborn joblib

# 2. Backend Web Framework
pip install Flask

# 3. Database Connector
pip install mysql-connector-python SQLAlchemy

# 4. C++ Integration (Optional but requested)
pip install pybind11
```

3 Step-by-Step Implementation Guide

3.1 Step 1: Fix & Train the Machine Learning Model

Your `ml_froud_detaction.ipynb` file is off to a great start, but there is a logic error. You are loading the CSV before checking if it exists.

Correction needed in your Notebook:

```
# REMOVE THIS LINE: data = pd.read_csv(data_path)
try:
    if os.path.exists(data_path):
        df = pd.read_csv(data_path)
        print("Dataset loaded successfully!")
```

```
# ... proceed with train_test_split and model training here ...

# After training, SAVE the model so Flask can use it:
# joblib.dump(model, 'fraud_model.pkl')
```

Your goal here is to train the model, test its accuracy, and then export it as a `.pkl` file.

3.2 Step 2: MySQL Database Setup

You need to install MySQL Server on your machine (or use a cloud provider like Aiven or PlanetScale). Create a database named `mazi_fraud_db` and create at least two tables:

- **Transactions Table:** columns for `id`, `user_id`, `amount`, `location`, `timestamp`.
- **Fraud Cases Table:** columns for `case_id`, `transaction_id`, `fraud_type`, `status`, `loss_amount`.

3.3 Step 3: Integrating C++ for High Performance

Since you want to include C++, you can create a file named `fast_math.cpp`. Write a function in C++ that calculates a complex risk score based on transaction velocity. Using `pybind11`, you compile this C++ file into a Python module (e.g., `import fast_math`) so your Flask app can run C++ code at lightning speed when processing thousands of transactions.

3.4 Step 4: Upgrading your Flask Backend (`app.py`)

Your current `app.py` has hardcoded dummy data. You need to upgrade it to:

1. Connect to MySQL using `mysql.connector`.
2. Load your trained AI model using `model = joblib.load('fraud_model.pkl')`.
3. Create an API endpoint (e.g., `/api/predict`) that takes incoming transaction data, passes it to the AI model, and returns whether it is fraudulent.
4. Replace the hardcoded `stats` dictionary by actually running `SELECT COUNT(*)` queries on your MySQL database.

3.5 Step 5: Building the Frontend Dashboard

In your `templates/dashboard.html` file:

- Use **HTML** for structure (tables, cards, navigation).
- Use **CSS** (or a framework like Bootstrap or Tailwind) to make it look professional, modern, and trustworthy (dark mode often looks great for security dashboards).
- Use **JavaScript** with a library like `Chart.js` to draw the trend graphs mentioned in your documentation. JavaScript will also be used to fetch real-time updates from your Flask backend without refreshing the page.

4 Recommendations & What to Add/Remove

4.1 What to Add

- **REST APIs:** Your documentation mentions ”Integration with external APIs”. Ensure your Flask app returns JSON data for certain routes so mobile apps or external systems can easily connect to MAZI.
- **Simulated Real-Time Stream:** Since this is a project, you won’t have real live bank data. Write a separate small Python script that inserts a random transaction into your MySQL database every 5 seconds to simulate a live environment!

4.2 What to Remove / Reconsider

- **Java (for Frontend):** Remove Java from your frontend plans. Stick entirely to JavaScript. It is the absolute industry standard and will make your life much easier when working with HTML/CSS.
- **Overcomplicating C++:** Don’t try to write your entire backend in C++. Keep Flask (Python) as the main brain, and only use C++ for one specific, mathematically heavy feature just to showcase your ability to integrate multiple languages.

5 Conclusion for Deployment

When you are ready to deploy online:

- Use **PythonAnywhere** or **Render.com** to host your Flask application for free.
- Use a managed MySQL database from a cloud provider.
- Make sure you never upload your database passwords to GitHub (use `.env` files!).

Good luck with your final exams! The scope of the MAZI dashboard is excellent and hits all the right marks for a top-tier computer science project.